

Rising Beyond Borders

Document 27 — Content Architecture Simplification & Staging Consolidation

Master Specification + Claude Code Implementation Prompt

Objective: Make the complete current website the normal working website. Remove unnecessary staging/demo content and image segregation while preserving security, payments, email, SEO, accessibility, release quality and infrastructure separation.

1. Target Architecture

- One normal source under the existing src/content architecture.
- One normal image/media source under the existing asset convention.
- npm run dev shows the complete current website.
- npm run build builds that same current website.
- No separate demo-content publishing/stripping pipeline.
- No staging-only image selection system.
- Infrastructure remains environment-separated for secrets, email, payments, analytics and host configuration.

Core principle: content is not an environment. Infrastructure is.

2. Remove / Simplify

- Separate staging/demo content tree used only to make staging different.
- Demo-content promotion/stripping logic whose sole purpose is environment separation.
- Staging-only image routing.
- Demo markers used only to identify ordinary working content.
- dev:staging/build:staging if their only purpose is content switching.
- Release rules that reject working content merely because it is currently temporary/demo.

3. Preserve

- Content models and reusable components.
- Malformed-record/reference/image validation.
- Accessibility and SEO validation.
- Security/CSP/secret scanning.
- Pre-rendering and hydration checks.
- Image optimization/responsive WebP.
- Deterministic release packaging and smoke tests.
- Resend and Razorpay server boundaries.
- Rate limiting and fail-closed production safeguards.
- Rollback and real-host deployment tooling.

4. New Working Workflow

- Developers edit src/content and normal image assets directly.

- Temporary/demo content is allowed during development/review.
- Before launch, temporary material is replaced with RBB-approved content.
- Run validation, accessibility, security, release and smoke checks.
- Freeze the approved release candidate and deploy.

5. Junior Developer Experience

- Open src/content to find website content.
- Open the normal asset location to find images.
- Run npm run dev to see the complete website.
- Edit content and refresh.
- Run npm run build to verify the same content in a production build.

6. Environment Separation That Remains

Concern	Development	Preview/Staging	Production
Content	Current content	Same current content	Final approved content
Images	Current assets	Same current assets	Final approved/licensed assets
Email	Off/test	Test	Live only if approved
Payments	Off/stand-in	Test/stand-in	Live only if approved
Analytics	Off	Off	Approved only
Secrets	Local	Host secrets	Production host secrets
Indexing	Off	Noindex	Approved routes

7. Production Gate Philosophy

The gate should ask whether content is **production-approved and safe to publish**, not whether the normal working content is called demo.

- Final release must contain no temporary labels, fictional names, example contact data or test credentials.
- Unverified impact/financial figures must remain held back or explicitly approved.
- Unknown/unlicensed images must be replaced or cleared.
- Staging robots/noindex and staging-only routes must not ship.
- Secrets, fixtures and unsafe URLs must never ship.

8. RBB Content Safety

- Mission/vision remain subject to RBB approval.
- Impact figures remain pending until verified.
- Financial percentages remain pending until verified.
- Team, projects, geography and stories require approved records.
- Contact information requires RBB-provided values.
- Policies require RBB/legal approval.
- Donation wording/settings require approval.
- Image ownership/licensing must be resolved before production.

9. Image Architecture

- The current 23 images can become the normal working image set.
- Remove special demo-image routing where it only separates environments.
- Keep responsive WebP/Picture implementation and alt text.
- Keep missing/broken/unsafe image validation.
- Replace temporary images with approved/licensed RBB images before launch.
- Components must not depend on a demo-image flag.

10. Preview Strategy

- Preview remains useful but previews the same content/build system.
- Preview URL remains noindex.
- Preview may use test email/payment configuration.
- Production credentials are not used merely because preview exists.
- Production later uses the same codebase after final content replacement.

11. Migration Safety

- Inventory all demo/staging-specific imports, flags, validators, scripts and environments.
- Separate pure content segregation from legitimate production safeguards.
- Migrate current staging content into normal content sources.
- Migrate current images into normal assets.
- Remove only logic whose sole purpose is content/environment segregation.
- Run all existing tests before and after.
- Do not redesign pages or delete reusable UI components.

12. Documentation

- Update CLAUDE.md and CONTENT_GUIDE.md for the single-source workflow.
- Update CONTENT_READINESS.md to describe final replacement/freeze.
- Retire or repurpose DEMO_CONTENT.md if it no longer represents architecture.
- Retain useful image inventory information without a staging-only dependency.
- Update deployment/release docs to remove assumptions of separate staging content.

13. Claude Code Implementation Prompt

Implement Document 27 — Content Architecture Simplification & Staging Consolidation for Rising Beyond Borders.

Treat Documents 1–26 and the repository as authoritative.

Core rule: Content is not an environment. Infrastructure is.

1. Audit all staging/demo-specific content sources, image sources, flags, build steps, validators and scripts.
2. Distinguish pure content segregation from legitimate security/release controls.
3. Migrate the current complete staging content into the normal src/content architecture.
4. Migrate the current 23 demo images into the normal asset/image architecture.
5. Make npm run dev show the complete current website.
6. Remove dev:staging/build:staging only if their sole purpose is content switching.
7. Remove demo stripping/publishing logic only where it exists solely for content separation.
8. Preserve content validation, SEO, accessibility, security, CSP, pre-rendering, image optimization, release packaging, smoke tests, Resend, Razorpay and rate-limit safeguards.
9. Change release logic from 'demo content must never enter production' to 'final production content must be approved before release'. Split development validation from final production-content readiness if needed.
10. Preserve preview noindex and production indexing controls.

11. Do not invent RBB claims or change page design.
12. Keep forms, donations and analytics gated exactly as they are.
13. Update documentation.
14. Run the complete existing test suite and compare before/after.
15. Do not stage or commit.

Final report: content/image sources before and after; removed staging logic; preserved safeguards; command changes; release-gate changes; documentation; tests; build; image checks; regressions; remaining RBB blockers.

Do not blindly delete files named demo. Inspect their purpose first.

14. Do Not Do

- Do not remove security controls.
- Do not expose Resend/Razorpay secrets.
- Do not enable live email/payments/analytics.
- Do not remove content validation entirely.
- Do not remove pre-rendering or image optimization.
- Do not redesign unrelated sections.
- Do not stage or commit.

15. Definition of Done

- One understandable content source and image source.
- npm run dev shows the complete current website.
- Normal build uses the same content.
- Content segregation is removed where unnecessary.
- Production safeguards remain.
- Release checks distinguish engineering correctness from final content approval.
- Existing critical tests pass.
- No production secrets introduced.
- No page redesign during migration.

16. Final Content Replacement

When RBB supplies final material, replace current working records/assets, validate them, run final production-content readiness checks, freeze the release candidate, then deploy. No second staging-to-production content migration should be required.

End of Document 27